

Project Planning Report: Optimisation Methods for Variational Inference

David Ewing (82171165)
ENEL445
University of Canterbury

2 May 2026

Background and Problem Intuition

Bayesian inference provides principled uncertainty quantification but requires computing intractable posterior distributions. For most real-world models, the posterior integral,

$$p(\boldsymbol{\theta} \mid \mathbf{y}) = \frac{p(\mathbf{y} \mid \boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathbf{y})},$$

cannot be evaluated analytically. Variational Inference (VI) converts this inference problem into an optimisation problem, making it directly amenable to the gradient-based methods in this course.

Instead of computing the true posterior, VI finds the “best” approximation $q(\boldsymbol{\theta})$ from a tractable family by maximising the Evidence Lower Bound (ELBO). This transforms intractable integration into tractable optimisation—I think this is a natural fit for an optimisation course project.

Key Notation

Symbol	Meaning
$\boldsymbol{\beta}$	Regression coefficient vector
$\mathbf{u}$	Random effects vector (hierarchical model)
$\boldsymbol{\varepsilon}$	Observation noise vector
τ_e	Observation-noise precision
τ_u	Random-effects precision
$\boldsymbol{\mu}_\beta$	Mean of variational posterior $q(\boldsymbol{\beta})$
$\boldsymbol{\Sigma}_\beta$	Covariance of variational posterior $q(\boldsymbol{\beta})$
$\mathbf{L}$	Lower-triangular Cholesky factor of $\boldsymbol{\Sigma}_\beta$
η_a, η_b	Log-space parameters for a_e, b_e

Optimisation Problem Formulation

Decision Variables

For Bayesian linear regression with p predictors, the variational parameters (decision variables) are:

- $\boldsymbol{\mu}_\beta \in \mathbb{R}^p$: posterior mean of regression coefficients
- $\boldsymbol{\Sigma}_\beta \in \mathbb{R}^{p \times p}$: posterior covariance¹ (symmetric positive definite²)
- $a_e, b_e > 0$: shape and rate parameters of the variational Gamma factor for precision

Here, a_e and b_e parameterise $q(\tau_e) = \text{Gamma}(a_e, b_e)$; the model prior on τ_e is specified in the Test Case section.

Total dimension: $d = p + \frac{p(p+1)}{2} + 2 = 11$ variables for $p = 3$ predictors.

Objective Function

Maximise the Evidence Lower Bound (ELBO):

$$\text{ELBO}(\boldsymbol{\phi}) = \mathbb{E}_q[\log p(\mathbf{y}, \boldsymbol{\beta}, \tau_e)] + H(q) \quad (1)$$

$$= \mathbb{E}_q[\log p(\mathbf{y}, \boldsymbol{\beta}, \tau_e)] - \mathbb{E}_q[\log q(\boldsymbol{\beta}, \tau_e)]. \quad (2)$$

where $\boldsymbol{\phi} = (\boldsymbol{\mu}_\beta, \boldsymbol{\Sigma}_\beta, a_e, b_e)$ are the variational parameters.

Constraints

- $\boldsymbol{\Sigma}_\beta \succ 0$: Covariance matrix must be positive definite (see also: positive semidefinite, $\mathbf{A} \succeq 0$ ³)
- $a_e, b_e > 0$: Gamma parameters must be positive
- Optional: Minimum variance constraints $\sigma_{\beta_j}^2 \geq \sigma_{\min}^2$ to prevent underdispersion

Constraint Handling via Reparameterisation:

To enforce $\boldsymbol{\Sigma}_\beta \succ 0$, use Cholesky decomposition:

$$\boldsymbol{\Sigma}_\beta = \mathbf{L}\mathbf{L}^T \quad (3)$$

where $\mathbf{L} \in \mathbb{R}^{p \times p}$ is lower triangular with positive diagonal entries. The unconstrained parameters are:

- $L_{ii} = e^{\ell_{ii}}$ for $i = 1, \dots, p$ (exponentiate to ensure positivity)

¹**Covariance** describes how much two variables move together. $\Sigma_{\beta,ij}$ tells you how much component β_i and β_j of the regression coefficients tend to vary together in the posterior. The diagonal entries $\Sigma_{\beta,ii}$ are the variances (spread) of each individual coefficient.

²A matrix $\mathbf{A}$ is **positive definite** if $\mathbf{v}^T \mathbf{A} \mathbf{v} > 0$ for every non-zero vector $\mathbf{v}$. Intuitively: the matrix amplifies every direction — there is no direction in which it squashes things to zero or flips them negative. For a covariance matrix this means every direction in parameter space has genuine positive uncertainty. Written $\mathbf{A} \succ 0$.

³**Positive semidefinite** ($\mathbf{A} \succeq 0$) allows $\mathbf{v}^T \mathbf{A} \mathbf{v} = 0$ for some non-zero $\mathbf{v}$, meaning some combination of coefficients has *exactly* zero uncertainty — the distribution is flat in that direction. A valid covariance matrix only needs to be positive semidefinite, but a semidefinite (not strictly definite) matrix is non-invertible⁴ and leads to degenerate directions.⁵

- $L_{ij} = \ell_{ij}$ for $i > j$ (off-diagonal entries unconstrained)

Total Cholesky parameters: $\frac{p(p+1)}{2}$ elements of ℓ .

To enforce $a_e, b_e > 0$, use log-space parameterisation:

$$a_e = e^{\eta_a}, \quad b_e = e^{\eta_b} \quad (4)$$

where $\eta_a, \eta_b \in \mathbb{R}$ are unconstrained.

Unconstrained parameter vector: $\xi = (\mu_\beta, \ell, \eta_a, \eta_b) \in \mathbb{R}^d$ with $d = p + \frac{p(p+1)}{2} + 2 = 11$ for $p = 3$.

Computational Characteristics

- Gradient computation: $O(np^2)$ per evaluation (dominated by matrix operations)
- Hessian computation: $O(np^2 + d^3)$ for Newton's method
- Parameters exhibit coupling (changing μ_β affects optimal Σ_β)
- Must handle special functions: digamma $\psi(\cdot)$, trigamma $\psi'(\cdot)$ for Gamma distribution

Test Case: Bayesian Linear Regression

The test case uses Bayesian linear regression with:

$$\mathbf{y} \sim \mathcal{N}(\mathbf{X}\boldsymbol{\beta}, \tau_e^{-1}\mathbf{I}_n) \quad (\text{likelihood}) \quad (5)$$

$$\boldsymbol{\beta} \sim \mathcal{N}(\mathbf{0}, \lambda_\beta^{-1}\mathbf{I}_p) \quad (\text{prior on coefficients}) \quad (6)$$

$$\tau_e \sim \text{Gamma}(\alpha_e, \gamma_e) \quad (\text{prior on precision}) \quad (7)$$

Mean-field approximation:

$$q(\boldsymbol{\beta}, \tau_e) = q(\boldsymbol{\beta}) q(\tau_e) \quad (8)$$

$$q(\boldsymbol{\beta}) = \mathcal{N}(\boldsymbol{\mu}_\beta, \boldsymbol{\Sigma}_\beta) \quad (9)$$

$$q(\tau_e) = \text{Gamma}(a_e, b_e) \quad (10)$$

Variational parameters: $\boldsymbol{\phi} = (\boldsymbol{\mu}_\beta, \boldsymbol{\Sigma}_\beta, a_e, b_e)$ with dimension $d = p + \frac{p(p+1)}{2} + 2$

For $p = 3$ predictors: $d = 3 + 6 + 2 = 11$ parameters to optimise.

Overview of Optimisation Methods

Methods considered

- CAVI: Domain-specific baseline exploiting problem structure
- Gradient Ascent: First-order baseline method using gradient information and line search
- Newton's Method: Second-order method with quadratic local convergence near a well-conditioned optimum
- BFGS: Quasi-Newton method that approximates Hessian information; practical general-purpose method for smooth unconstrained problems

Baseline Method: CAVI⁶

Coordinate Ascent Variational Inference (CAVI) is VI-specific. It updates each variational parameter block sequentially using closed-form solutions:

$$\boldsymbol{\Sigma}_\beta \leftarrow \left(\frac{a_e}{b_e} \mathbf{X}^T \mathbf{X} + \lambda_\beta \mathbf{I}_p \right)^{-1} \quad (11)$$

$$\boldsymbol{\mu}_\beta \leftarrow \frac{a_e}{b_e} \boldsymbol{\Sigma}_\beta \mathbf{X}^T \mathbf{y} \quad (12)$$

$$a_e \leftarrow \alpha_e + \frac{n}{2} \quad (13)$$

$$b_e \leftarrow \gamma_e + \frac{1}{2} [\|\mathbf{y} - \mathbf{X}\boldsymbol{\mu}_\beta\|^2 + \text{tr}(\mathbf{X}^T \mathbf{X} \boldsymbol{\Sigma}_\beta)] \quad (14)$$

Properties: Monotonic ELBO increase⁷, linear convergence, $O(np^2)$ per iteration. Provides analytical benchmark for validation.

⁶Blei, Kucukelbir, and McAuliffe (2017), Variational Inference: A Review for Statisticians (Journal of the American Statistical Association).

⁷Monotonic ELBO increase means the objective is non-decreasing across iterations: $\mathcal{L}^{(t+1)} \geq \mathcal{L}^{(t)}$. In CAVI, each coordinate update maximises the ELBO with respect to one variational factor while holding the others fixed, so each step cannot decrease the ELBO.

Mapping ENEL445 Methods to VI Problem

1. Parameter Ascent (Course: gradient descent)

- Standard iterative update: $\phi^{(t+1)} = \phi^{(t)} + \alpha_t \nabla \text{ELBO}(\phi^{(t)})$
- Line search from Chapter 3.2 (bisection/Wolfe conditions)
- Stopping criteria: $\|\nabla \text{ELBO}\| < \epsilon$
- Convergence characteristics: linear rate expected for strongly concave ELBO

2. Newton's Method

- Standard Newton update: $\phi^{(t+1)} = \phi^{(t)} + \alpha_t [\nabla^2 \text{ELBO}]^{-1} \nabla \text{ELBO}$
- Requires computing and inverting the Hessian matrix
- Computational cost: $O(d^3)$ per iteration for Hessian inversion
- Convergence characteristics: quadratic convergence near optimum (when Hessian is negative definite)
- Challenge: Hessian may not be negative definite away from the optimum

3. BFGS Quasi-Newton (one of five required methods from Project 2)

- BFGS update formula builds inverse Hessian approximation
- Uses gradient differences: $\mathbf{s}_k = \phi^{(k)} - \phi^{(k-1)}$, $\mathbf{y}_k = \nabla \text{ELBO}(\phi^{(k)}) - \nabla \text{ELBO}(\phi^{(k-1)})$
- Computational cost: $O(d^2)$ per iteration
- Convergence characteristics: superlinear convergence expected

Comparison will reveal whether general optimisation methods can compete with or outperform problem-specific algorithms.

Solution Plan

Phase 1: Mathematical Foundations — COMPLETE

Status: All mathematical derivations and implementation specifications verified and documented.

- ✓ Complete ELBO derivation for Bayesian linear regression
- ✓ CAVI closed-form update equations with matrix inversion lemma
- ✓ Gradient expressions: ∇_{μ_β} ELBO, ∇_{Σ_β} ELBO, ∇_{a_e} ELBO, ∇_{b_e} ELBO
- ✓ Gradient transformation for Cholesky parameterisation: ∇_ℓ ELBO
- ✓ Gradient transformation for log-space parameterisation: ∇_{η_a} ELBO, ∇_{η_b} ELBO
- ✓ Hessian block structure: diagonal and mixed blocks
- ✓ BFGS update formulation with Woodbury identity
- ✓ Initialisation strategy with principled defaults
- ✓ Line search algorithm specifications (Armijo backtracking)
- ✓ Newton regularisation strategy for indefinite Hessian
- ✓ Convergence criteria with multiple stopping conditions

All derivations documented in `vi_optimisation_solution.pdf` with complete algebraic steps.

Phase 2: Implementation

1. Data Generation (15%)

- Simulate data: $n = 100$ observations, $p = 3$ predictors
- Generate $\mathbf{X}$ from standard normal, $\beta_{\text{true}} = [1, -0.5, 2]^T$
- Add noise: $\epsilon \sim \mathcal{N}(0, 0.5^2)$
- Store data in NumPy arrays for efficient computation

2. Baseline Implementation: CAVI (40%)

- Implement closed-form coordinate updates
- Track ELBO convergence
- Stopping criterion: $|\text{ELBO}^{(t+1)} - \text{ELBO}^{(t)}| < 10^{-6}$
- Numerical gradient check against analytical CAVI updates

3. Method 1: Gradient Ascent (40%)

- Implement ELBO gradient computation with Cholesky and log-space transforms
- Line search: Armijo backtracking (Algorithm 1 in Implementation Specifications)
- Stopping criterion: $\|\nabla \text{ELBO}\| < 10^{-6}$ or relative ELBO change $< 10^{-8}$

- Track iteration count and ELBO values
4. Method 2: Newton's Method (25%)
 - Implement Hessian computation (block structure)
 - Regularisation: add $\lambda \mathbf{I}$ with $\lambda = \max(10^{-6}, -2\lambda_{\min})$ if Hessian not negative definite
 - Line search for step size α_t
 - Track convergence and computational cost
 5. Method 3: BFGS Quasi-Newton (30%)
 - Initialise inverse Hessian approximation: $\mathbf{B}_0 = \mathbf{I}_d$
 - Implement BFGS update with curvature condition check
 - Track superlinear convergence
 6. Comparison and Analysis (20%)
 - Compare convergence speed (iterations to 10^{-6} tolerance)
 - Compare computational cost (time per iteration, total time)
 - Compare solution quality (posterior approximation accuracy vs MCMC)
 - Analyse when each method performs best
 - Generate convergence plots and performance tables

Constraint Handling via Reparameterisation

The variational parameters are not free. Σ_β must remain symmetric positive definite throughout optimisation, and a_e, b_e must remain strictly positive. Four strategies exist for handling these constraints in a gradient-based optimiser: ignore them (the algorithm crashes when $\log |\Sigma_\beta|$ becomes undefined); add a penalty term (introduces a tuning parameter and ill-conditioning as $\rho \rightarrow \infty$); project back to the feasible set after each step (costs $O(p^3)$ per iteration and is non-differentiable at the boundary, breaking BFGS); or reparameterise so the constraints are encoded structurally and cannot be violated. This work uses reparameterisation throughout—it requires no tuning parameters, no outer loops, and produces a smooth unconstrained problem fully compatible with gradient ascent, Newton, and BFGS. Two transformations are applied: one for the matrix constraint on Σ_β , and one for the scalar positivity constraints on a_e and b_e .

Cholesky Reparameterisation

Σ_β is the covariance matrix of the variational distribution $q(\beta) = \mathcal{N}(\mu_\beta, \Sigma_\beta)$, and must therefore remain symmetric positive definite (SPD) at every iteration. A gradient step has no awareness of this constraint: nothing stops an update from producing a matrix with a negative eigenvalue, after which $\log |\Sigma_\beta|$ and Σ_β^{-1} are undefined and the algorithm crashes.

The transformation is $\Sigma_\beta = \mathbf{L}\mathbf{L}^T$, where $\mathbf{L}$ is lower triangular. Optimising over $\mathbf{L}$ instead of Σ_β directly ensures any $\mathbf{L}$ produces a valid symmetric positive semidefinite matrix by construction. However, a residual constraint remains: the diagonal entries ℓ_{ii} must be strictly positive for Σ_β to be positive definite rather than merely positive semidefinite. Three options exist: ignore the diagonal constraint (positivity can still be violated, recovering the original problem); apply a softplus function $\ell_{ii} = \log(1 + e^{u_i})$ (valid but less standard); or apply a log-space transformation $\ell_{ii} = e^{\tilde{\ell}_{ii}}$, keeping $\tilde{\ell}_{ii} \in \mathbb{R}$ unconstrained. The log-space option is used here. The parameter count is unchanged at $p(p+1)/2$ free entries: p log-diagonal entries $\tilde{\ell}_{ii}$ and $p(p-1)/2$ unconstrained off-diagonal entries ℓ_{ij} , $i > j$.

Log-Space Reparameterisation

The Gamma shape and rate parameters $a_e, b_e > 0$ carry a scalar positivity constraint. The same reparameterisation principle applies: define $\eta_a = \log a_e$ and $\eta_b = \log b_e$, so that $a_e = e^{\eta_a}$ and $b_e = e^{\eta_b}$. Since the exponential function maps $\mathbb{R}$ onto $(0, \infty)$, the optimiser moves η_a, η_b freely over the real line and the positivity constraint is structurally impossible to violate. This is the scalar analogue of the Cholesky diagonal transformation, and the same principle as substituting $x = e^t$ to eliminate $x > 0$ in single-variable optimisation.

After both transformations, the effective optimisation variables are $(\mu_\beta, \tilde{\ell}_{11}, \ell_{21}, \tilde{\ell}_{22}, \dots, \eta_a, \eta_b)$ —all unconstrained reals. The quantities Σ_β, a_e, b_e are recovered on demand when evaluating the ELBO and its gradients.

Implementation Specifications

This section provides complete algebraic detail for all numerical transformations required for implementation. These specifications ensure the final report can reference exact formulas without derivation.

1. Cholesky Parameterisation

Forward transformation (constrained to unconstrained):

Given $\Sigma_\beta \succ 0$, compute Cholesky decomposition:

$$\Sigma_\beta = \mathbf{L}\mathbf{L}^T \quad (15)$$

where $\mathbf{L}$ is lower triangular with $L_{ii} > 0$.

Extract unconstrained parameters:

$$\ell_{ii} = \log L_{ii} \quad \text{for } i = 1, \dots, p \quad (16)$$

$$\ell_{ij} = L_{ij} \quad \text{for } i > j \quad (17)$$

Inverse transformation (unconstrained to constrained):

Given unconstrained vector ℓ , reconstruct $\mathbf{L}$:

$$L_{ii} = e^{\ell_{ii}} \quad \text{for } i = 1, \dots, p \quad (18)$$

$$L_{ij} = \ell_{ij} \quad \text{for } i > j \quad (19)$$

$$L_{ij} = 0 \quad \text{for } i < j \quad (20)$$

Then compute:

$$\Sigma_\beta = \mathbf{L}\mathbf{L}^T \quad (21)$$

Gradient transformation (chain rule):

Let $\mathcal{L}(\xi)$ be the ELBO expressed in unconstrained parameters. By the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \ell_{ij}} = \sum_{k,m} \frac{\partial \mathcal{L}}{\partial \Sigma_{\beta,km}} \frac{\partial \Sigma_{\beta,km}}{\partial \ell_{ij}} \quad (22)$$

For diagonal entries ($i = j$):

$$\frac{\partial \Sigma_{\beta,km}}{\partial \ell_{ii}} = \frac{\partial}{\partial \ell_{ii}} \left[\sum_s L_{ks} L_{ms} \right] \quad (23)$$

$$= \frac{\partial L_{ki}}{\partial \ell_{ii}} L_{mi} + L_{ki} \frac{\partial L_{mi}}{\partial \ell_{ii}} \quad (24)$$

Since $L_{ii} = e^{\ell_{ii}}$ and $L_{ij} = \ell_{ij}$ for $i \neq j$:

$$\frac{\partial L_{ki}}{\partial \ell_{ii}} = \begin{cases} e^{\ell_{ii}} = L_{ii} & \text{if } k = i \\ 0 & \text{if } k \neq i \end{cases} \quad (25)$$

Therefore:

$$\frac{\partial \Sigma_{\beta, km}}{\partial \ell_{ii}} = \delta_{ki} L_{ii} L_{mi} + \delta_{mi} L_{ki} L_{ii} \quad (26)$$

$$= L_{ii} (\delta_{ki} L_{mi} + \delta_{mi} L_{ki}) \quad (27)$$

For off-diagonal Cholesky entries ($i > j$):

$$\frac{\partial L_{ki}}{\partial \ell_{ij}} = \delta_{ki} \delta_{ji} = \begin{cases} 1 & \text{if } k = i, s = j \\ 0 & \text{otherwise} \end{cases} \quad (28)$$

Thus:

$$\frac{\partial \Sigma_{\beta, km}}{\partial \ell_{ij}} = \delta_{ki} L_{mj} + \delta_{mi} L_{kj} \quad (29)$$

Complete gradient formula:

Let $\mathbf{G} = \nabla_{\Sigma_{\beta}} \mathcal{L}$ be the gradient with respect to the covariance matrix. Then:

$$\frac{\partial \mathcal{L}}{\partial \ell_{ii}} = L_{ii} \sum_{k,m} G_{km} (\delta_{ki} L_{mi} + \delta_{mi} L_{ki}) \quad (30)$$

$$= L_{ii} \left(2 \sum_m G_{im} L_{mi} - G_{ii} L_{ii} \right) \quad (31)$$

$$= 2L_{ii} (\mathbf{GL})_{ii} - L_{ii}^2 G_{ii} \quad (32)$$

$$= L_{ii} (2(\mathbf{GL})_{ii} - L_{ii} G_{ii}) \quad (33)$$

For off-diagonal:

$$\frac{\partial \mathcal{L}}{\partial \ell_{ij}} = \sum_{k,m} G_{km} (\delta_{ki} L_{mj} + \delta_{mi} L_{kj}) \quad (34)$$

$$= \sum_m G_{im} L_{mj} + \sum_k G_{km} L_{kj} \quad (35)$$

$$= (\mathbf{GL})_{ij} + (\mathbf{G}^T \mathbf{L})_{ij} \quad (36)$$

$$= ((\mathbf{G} + \mathbf{G}^T) \mathbf{L})_{ij} \quad (37)$$

Implementation: Compute $\mathbf{G} = \nabla_{\Sigma_{\beta}} \text{ELBO}$, then transform to Cholesky space using the above formulas.

2. Log-Space Parameterisation

Forward transformation:

$$\eta_a = \log a_e \quad (38)$$

$$\eta_b = \log b_e \quad (39)$$

Inverse transformation:

$$a_e = e^{\eta_a} \quad (40)$$

$$b_e = e^{\eta_b} \quad (41)$$

Gradient transformation with Jacobian adjustment:

By the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \eta_a} = \frac{\partial \mathcal{L}}{\partial a_e} \frac{\partial a_e}{\partial \eta_a} = \frac{\partial \mathcal{L}}{\partial a_e} \cdot e^{\eta_a} = a_e \cdot \frac{\partial \mathcal{L}}{\partial a_e} \quad (42)$$

Similarly:

$$\frac{\partial \mathcal{L}}{\partial \eta_b} = b_e \cdot \frac{\partial \mathcal{L}}{\partial b_e} \quad (43)$$

Implementation: Compute gradients $\nabla_{a_e} \text{ELBO}$ and $\nabla_{b_e} \text{ELBO}$, then multiply by a_e and b_e respectively.

3. Initialisation Strategy

Primary initialisation (from OLS):

$$\boldsymbol{\mu}_\beta^{(0)} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (\text{OLS estimate}) \quad (44)$$

$$\boldsymbol{\Sigma}_\beta^{(0)} = \mathbf{I}_p \quad (\text{moderate initial uncertainty}) \quad (45)$$

$$a_e^{(0)} = \alpha_e + \frac{n}{2} \approx 51 \quad (\text{data-informed shape}) \quad (46)$$

$$b_e^{(0)} = 1.0 \quad (\text{vague initial rate}) \quad (47)$$

Rationale: OLS provides good starting point for $\boldsymbol{\mu}_\beta$; identity covariance avoids overconfidence; a_e reflects data contribution.

Alternative initialisation (zero-centred):

$$\boldsymbol{\mu}_\beta^{(0)} = \mathbf{0}_p \quad (48)$$

$$\boldsymbol{\Sigma}_\beta^{(0)} = (\mathbf{X}^T \mathbf{X})^{-1} \quad (\text{frequentist uncertainty}) \quad (49)$$

$$a_e^{(0)} = \alpha_e + 1 = 2 \quad (50)$$

$$b_e^{(0)} = \gamma_e + 1 = 2 \quad (51)$$

Cholesky initialisation:

Given initial $\boldsymbol{\Sigma}_\beta^{(0)}$, compute Cholesky:

$$\boldsymbol{\Sigma}_\beta^{(0)} = \mathbf{L}^{(0)} (\mathbf{L}^{(0)})^T \quad (52)$$

$$\ell_{ii}^{(0)} = \log L_{ii}^{(0)} \quad (53)$$

$$\ell_{ij}^{(0)} = L_{ij}^{(0)} \quad \text{for } i > j \quad (54)$$

Log-space initialisation:

$$\eta_a^{(0)} = \log a_e^{(0)} \quad (55)$$

$$\eta_b^{(0)} = \log b_e^{(0)} \quad (56)$$

4. Line Search: Armijo Backtracking

Algorithm 1 Armijo Backtracking Line Search

Require: Current point $\xi^{(t)}$, search direction $d^{(t)}$, gradient $\nabla \mathcal{L}^{(t)}$

Require: Parameters: $c_1 = 10^{-4}$ (sufficient decrease), $\rho = 0.5$ (backtracking factor), $\alpha_{\max} = 1.0$

Ensure: Step size α_t satisfying Armijo condition

```

1:  $\alpha \leftarrow \alpha_{\max}$ 
2:  $\text{max\_iter} \leftarrow 50$ 
3:  $k \leftarrow 0$ 
4: while  $k < \text{max\_iter}$  do
5:   Compute trial point:  $\xi_{\text{trial}} \leftarrow \xi^{(t)} + \alpha d^{(t)}$ 
6:   Evaluate:  $\mathcal{L}_{\text{trial}} \leftarrow \mathcal{L}(\xi_{\text{trial}})$ 
7:   Compute sufficient decrease threshold:  $\mathcal{L}_{\text{threshold}} \leftarrow \mathcal{L}(\xi^{(t)}) + c_1 \alpha (\nabla \mathcal{L}^{(t)})^T d^{(t)}$ 
8:   if  $\mathcal{L}_{\text{trial}} \geq \mathcal{L}_{\text{threshold}}$  then
9:     return  $\alpha$  // Armijo condition satisfied
10:  end if
11:   $\alpha \leftarrow \rho \alpha$  // Reduce step size
12:   $k \leftarrow k + 1$ 
13: end while
14: Warning: Line search failed, return  $\alpha = 10^{-8}$  (damped step)
15: return  $\alpha$ 

```

Key parameters:

- $c_1 = 10^{-4}$: Standard value for sufficient decrease (Nocedal & Wright)
- $\rho = 0.5$: Halves step size at each backtrack iteration
- $\alpha_{\max} = 1.0$: Start with full Newton step (for Newton and BFGS)
- For gradient ascent: may use $\alpha_{\max} = 0.1$ initially

5. Newton's Method Regularisation

When Hessian $\mathbf{H} = \nabla^2 \text{ELBO}$ is not negative definite (for maximisation), regularise:

$$\mathbf{H}_{\text{reg}} = \mathbf{H} - \lambda \mathbf{I}_d \quad (57)$$

Regularisation parameter selection:

1. Compute eigenvalue⁸ decomposition: $\mathbf{H} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^T$ (expensive, $O(d^3)$)
2. Or use Cholesky attempt: try $\mathbf{H} = \mathbf{R}^T \mathbf{R}$; if fails, Hessian not positive definite

⁸**Eigenvalues** are numbers $\lambda_1, \lambda_2, \dots$ that describe how a matrix scales space along its natural axes (eigenvectors). For a symmetric matrix like Σ_β : all eigenvalues > 0 means positive definite; all eigenvalues ≥ 0 means positive semidefinite; any eigenvalue < 0 means not a valid covariance matrix. They are the stretching factors of the matrix in each principal direction.

3. Simpler approach (Levenberg damping):

$$\lambda = \begin{cases} 0 & \text{if Cholesky succeeds (Hessian negative definite)} \\ \max(10^{-6}, -2\lambda_{\min}) & \text{otherwise} \end{cases} \quad (58)$$

where $\lambda_{\min}$ is smallest eigenvalue of $\mathbf{H}$.

Practical implementation (without eigenvalue computation):

Start with $\lambda = 0$. If Cholesky of $-\mathbf{H}$ fails:

$$\lambda \leftarrow 10^{-4} \quad (59)$$

$$\mathbf{H}_{\text{reg}} \leftarrow \mathbf{H} - \lambda \mathbf{I}_d \quad (60)$$

Repeat with $\lambda \leftarrow 10\lambda$ until Cholesky succeeds or $\lambda > 10^3$ (failure).

Newton search direction:

$$\mathbf{d}^{(t)} = -\mathbf{H}_{\text{reg}}^{-1} \nabla \mathcal{L}^{(t)} \quad (61)$$

For maximisation, this should be an ascent direction: $(\nabla \mathcal{L})^T \mathbf{d} > 0$.

6. Convergence Criteria

Implement multiple stopping conditions (logical OR):

1. **Gradient norm:** $\|\nabla \mathcal{L}^{(t)}\|_2 < \epsilon_g = 10^{-6}$

Indicates first-order optimality (stationary point reached).

2. **Relative ELBO change:**

$$\frac{|\mathcal{L}^{(t)} - \mathcal{L}^{(t-1)}|}{1 + |\mathcal{L}^{(t-1)}|} < \epsilon_{\text{rel}} = 10^{-8} \quad (62)$$

Prevents premature termination when ELBO is large in magnitude.

3. **Parameter change:**

$$\|\boldsymbol{\xi}^{(t)} - \boldsymbol{\xi}^{(t-1)}\|_2 < \epsilon_x = 10^{-6} \quad (63)$$

Catches slow convergence in parameter space.

4. **Maximum iterations:** $t \geq t_{\max} = 1000$

Safety check to prevent infinite loops.

Convergence flag assignment:

- Status 0: Gradient norm criterion (ideal)
- Status 1: Relative ELBO change (acceptable)
- Status 2: Parameter change (acceptable but slower)
- Status 3: Maximum iterations (failure to converge)

7. Numerical Gradient Verification

For debugging, verify analytical gradients using finite differences:

Central difference formula:

$$\frac{\partial \mathcal{L}}{\partial \xi_i} \approx \frac{\mathcal{L}(\boldsymbol{\xi} + h\mathbf{e}_i) - \mathcal{L}(\boldsymbol{\xi} - h\mathbf{e}_i)}{2h} \quad (64)$$

where $\mathbf{e}_i$ is the i -th standard basis vector and h is step size.

Step size selection:

For float64 precision ($\epsilon_{\text{machine}} \approx 2.2 \times 10^{-16}$):

$$h = \sqrt{\epsilon_{\text{machine}}} \max(1, |\xi_i|) \approx 1.5 \times 10^{-8} \max(1, |\xi_i|) \quad (65)$$

Error metric:

$$\text{error}_i = \frac{|g_i^{\text{analytical}} - g_i^{\text{numerical}}|}{\max(|g_i^{\text{analytical}}|, |g_i^{\text{numerical}}|, 10^{-8})} \quad (66)$$

Acceptable if $\text{error}_i < 10^{-5}$ for all i .

8. Failure Mode Handling

Common failure modes and recovery:

1. Matrix not positive definite:

- Detection: Cholesky decomposition fails
- Recovery: Add regularisation $\boldsymbol{\Sigma}_\beta \leftarrow \boldsymbol{\Sigma}_\beta + 10^{-6} \mathbf{I}_p$

2. Line search exhaustion:

- Detection: 50 backtracking iterations without Armijo satisfaction
- Recovery: Accept tiny step $\alpha = 10^{-8}$ and continue; if recurring, terminate

3. ELBO decrease:

- Detection: $\mathcal{L}^{(t)} < \mathcal{L}^{(t-1)}$ after line search
- Recovery: Reduce step size aggressively or revert to previous parameters

4. Parameter divergence:

- Detection: $\|\boldsymbol{\xi}^{(t)}\|_2 > 10^{10}$ or any $|\xi_i| > 10^6$
- Recovery: Reinitialise from OLS estimates

5. NaN/Inf in ELBO or gradient:

- Detection: `np.isnan()` or `np.isinf()` checks
- Recovery: Reduce step size or add numerical safeguards (min/max clipping)

Success Metrics

1. Convergence: All methods reach ELBO optimum within tolerance
2. Correctness:
 - CAVI correctness: Gradient/Newton/BFGS implementation is correct if it reaches the same ELBO optimum / variational fixed point as CAVI (same VI objective, same variational family)
 - MCMC correctness: VI is assessed against MCMC as a posterior-accuracy benchmark (means, intervals, calibration), not expected to match exactly because VI is approximate
3. Performance hierarchy:
 - CAVI likely fastest for this conjugate problem
 - BFGS expected to be competitive general-purpose method
 - Newton may struggle with indefinite Hessian
 - MCMC/Gibbs expected slowest computationally, but best for posterior uncertainty accuracy (gold-standard benchmark, not a direct ELBO-optimiser competitor)
4. Numerical verification:
 - Analytical gradients match finite differences within 10^{-5} relative error
 - All methods reach same ELBO value (within 10^{-4})
 - Posterior means agree with CAVI and MCMC within 2 standard errors

Progress To Date (as of 1 April 2026)

Completed Work

1. **Problem formulation:** Optimisation problem fully specified with decision variables, objective function (ELBO), constraints, and reparameterisation strategy (Cholesky + log-space).
2. **Mathematical derivations — COMPLETE:**
 - ✓ Complete ELBO derivation from first principles (13 pages, documented)
 - ✓ CAVI algorithm with closed-form updates and convergence proof
 - ✓ Analytical gradient expressions for all 11 variational parameters
 - ✓ Gradient transformations for Cholesky and log-space parameterisations with complete chain-rule algebra
 - ✓ Hessian block structure for Newton's method
 - ✓ BFGS quasi-Newton formulation with update formulas
 - ✓ Penalty method formulation for constrained optimisation (alternative approach)
 - ✓ Initialisation strategy with OLS-based and zero-centred alternatives
 - ✓ Line search algorithm specification (Armijo backtracking with pseudocode)
 - ✓ Newton regularisation for indefinite Hessian
 - ✓ Convergence criteria with four stopping conditions

- ✓ Numerical gradient verification procedure
- ✓ Failure mode analysis with recovery strategies

All derivations documented in `vi_optimisation_solution.pdf` with complete algebraic steps suitable for reference in final report.

3. **Implementation readiness:** All mathematical specifications complete with step-by-step formulas. Ready to proceed with coding phase.

Preliminary Results

Mathematical analysis confirms:

- Problem is well-posed with unique global maximum (ELBO is concave in each coordinate block)
- CAVI updates guaranteed to converge monotonically
- Gradient-based methods require careful parameterisation to handle constraints
- Newton's method may require regularisation away from optimum
- BFGS expected to be robust general-purpose method

Next Steps

Week 7 (24-31 March):

- Implement CAVI baseline in Python with ELBO tracking
- Implement numerical gradient checker
- Verify CAVI gradients against finite differences
- Begin gradient ascent implementation

Week 8 (1-7 April):

- Complete gradient ascent with Armijo line search
- Implement Newton's method with regularisation
- Begin BFGS implementation

Week 9 (8-14 April):

- Complete BFGS implementation
- Run convergence comparisons on test case
- Generate convergence plots (ELBO vs iteration, time vs iteration)

Week 10 (15-21 April):

- Sensitivity analysis: different initialisations, data sizes
- MCMC validation (Gibbs sampler for ground truth)
- Performance profiling and optimisation

Week 11 (22-28 April):

- Final empirical comparison
- Generate all figures and tables for report
- Draft results section

Week 12 (May):

- Complete final report with performance analysis
- Review and polish all sections
- Submit by deadline

Alignment with Course Requirements

The VI problem represents real computational challenges, making the comparison of optimisation methods valuable, relevant, and interesting. The project implements BFGS (one of the five required methods from Project 2 specification) along with two additional methods (Newton, gradient ascent) for comprehensive comparison.

Mathematical specifications are now complete and documented with full algebraic detail, enabling implementation to proceed systematically with clear validation checkpoints.

Timeline

Week	Dates	Milestone
7	24-31 Mar	CAVI baseline + gradient verification
8	1-7 Apr	Gradient ascent + Newton's method
9	8-14 Apr	BFGS + convergence analysis
10	15-21 Apr	MCMC validation + sensitivity analysis
11	22-28 Apr	Final comparison + figure generation
12	May	Final report completion and submission

Cholesky Reparameterisation: Structural Constraint Satisfaction

The Problem with Direct Optimisation

The covariance matrix Σ_β must satisfy $\Sigma_\beta \succ 0$ (symmetric positive definite) at every iteration. If the optimiser updates Σ_β directly, a gradient step of the form

$$\Sigma_\beta^{(t+1)} = \Sigma_\beta^{(t)} + \alpha_t \nabla_{\Sigma_\beta} \mathcal{L} \quad (67)$$

can produce a matrix that is *not* positive definite. Standard gradient-based methods operate on unconstrained Euclidean space and have no built-in mechanism to respect the positive-definiteness constraint — the constraint is simply violated as a by-product of the update, leading to numerical failure (negative variances, failed Cholesky in later steps, NaN in the ELBO).

The Cholesky Solution: Structural Guarantee

Cholesky decomposition eliminates the constraint by parameterising Σ_β in terms of a lower-triangular factor $\mathbf{L}$:

$$\Sigma_\beta = \mathbf{L}\mathbf{L}^T, \quad L_{ii} > 0 \quad \forall i \quad (68)$$

The fundamental insight is that **every** lower-triangular matrix $\mathbf{L}$ with strictly positive diagonal entries produces a valid symmetric positive definite matrix:

- **Symmetry:** $(\mathbf{L}\mathbf{L}^T)^T = (\mathbf{L}^T)^T \mathbf{L}^T = \mathbf{L}\mathbf{L}^T$ — automatically symmetric.
- **Positive definiteness:** For any non-zero $\mathbf{v} \in \mathbb{R}^p$,

$$\mathbf{v}^T \mathbf{L}\mathbf{L}^T \mathbf{v} = \|\mathbf{L}^T \mathbf{v}\|^2 > 0 \quad (69)$$

provided $\mathbf{L}^T$ has full column rank, which holds whenever all $L_{ii} > 0$ (since $\mathbf{L}$ is lower triangular, its rank equals the number of non-zero diagonal entries).

It is therefore *structurally impossible* to produce an invalid Σ_β by updating $\mathbf{L}$, regardless of how large or in what direction the gradient step is taken. The constraint is not enforced by a penalty or projection — it is encoded into the *form* of the parameterisation itself.

Unconstrained Optimisation Over ℓ

The remaining difficulty is that $L_{ii} > 0$ is still an inequality constraint. This is resolved by log-parameterising the diagonal:

$$L_{ii} = e^{\ell_{ii}}, \quad \ell_{ii} \in \mathbb{R} \quad (\text{diagonal}) \quad (70)$$

$$L_{ij} = \ell_{ij}, \quad \ell_{ij} \in \mathbb{R} \quad (\text{lower off-diagonal}) \quad (71)$$

Since $e^{\ell_{ii}} > 0$ for all $\ell_{ii} \in \mathbb{R}$, the optimisation is now over a *fully unconstrained* parameter vector $\ell \in \mathbb{R}^{p(p+1)/2}$. No projection, no clipping, no penalty — any point in $\mathbb{R}^{p(p+1)/2}$ maps to a valid $\mathbf{L}$, and hence a valid $\Sigma_\beta = \mathbf{L}\mathbf{L}^T$.

Analogy with Log-Space Parameterisation

This is the same idea used for the Gamma parameters $a_e, b_e > 0$: instead of constraining $a_e > 0$ directly, we optimise over unconstrained $\eta_a = \log a_e \in \mathbb{R}$ and recover $a_e = e^{\eta_a}$, which is always positive. Cholesky + log-diagonal does the same thing for the cone of symmetric positive definite matrices: it provides a smooth bijection between unconstrained Euclidean space and the constraint set, so standard optimisation machinery applies without modification.

Why This Is Not in the Course Notes

Cholesky reparameterisation is a *numerical implementation technique*, not a course-level optimisation concept. The course teaches the optimisation methods (gradient ascent, Newton, BFGS); Cholesky is the domain-specific choice of how to handle the manifold structure of the constraint set $\Sigma_\beta \succ 0$ when applying those methods to this problem. It is standard practice in variational inference and Bayesian computation.

Reference

Bishop, C.M. (2006). *Pattern Recognition and Machine Learning*. Springer.

- Section 2.3 (The Gaussian Distribution) — symmetric positive definite covariance matrices and their properties.
- Appendix C (Properties of Matrices) — Cholesky factorisation and positive definite matrices.

Workspace copy available at: [reference/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf](#)

VB Posterior Variance Collapse

Observation

Comparison of the VB posterior against the MCMC (Gibbs sampler) gold standard reveals that the mean-field VB approximation systematically underestimates posterior variance — a well-known failure mode of mean-field variational inference.

This occurs because mean-field VB minimises the inclusive KL divergence:

$$\text{KL}[q||p] = \int q(\boldsymbol{\theta}) \log \frac{q(\boldsymbol{\theta})}{p(\boldsymbol{\theta} | \mathbf{y})} d\boldsymbol{\theta} \quad (72)$$

This direction penalises q for placing mass where p is small, but *does not* penalise q for missing mass where p is large. The result is a q that is narrower (overconfident) relative to the true posterior, particularly in curved or multi-modal posteriors.

Three Approaches to Prevent Collapse

1. Informative Prior on τ_e

Set non-zero prior hyperparameters (α_e, γ_e) to place a lower bound on the posterior variance. A tight prior on τ_e prevents the precision from being driven too high, which in turn keeps Σ_β from collapsing.

Mechanism: The CAVI update for b_e is:

$$b_e \leftarrow \gamma_e + \frac{1}{2} [\|\mathbf{y} - \mathbf{X}\boldsymbol{\mu}_\beta\|^2 + \text{tr}(\mathbf{X}^T \mathbf{X} \Sigma_\beta)] \quad (73)$$

With $\gamma_e > 0$, b_e has a minimum floor, bounding $\mathbb{E}[\tau_e] = a_e/b_e$ from above.

Limitation: Changes the model; requires prior elicitation.

2. Variance Lower Bound Constraint

After each CAVI update, clip Σ_β so its diagonal never falls below a minimum threshold $\sigma_{\min}^2$:

$$\Sigma_{\beta,jj} \leftarrow \max(\Sigma_{\beta,jj}, \sigma_{\min}^2) \quad \forall j \quad (74)$$

This was already noted in the Constraints section as an optional minimum variance constraint. In code:

```
Sigma_diag = np.diag(self.Sigma_beta)
np.fill_diagonal(self.Sigma_beta,
    np.maximum(Sigma_diag, sigma_min))
```

Limitation: Ad hoc; does not arise from a principled objective modification.

3. Entropy Regularisation of the ELBO

Add a regularisation term proportional to the entropy $H[q]$ of the variational distribution to the objective:

$$\mathcal{L}_{\text{reg}}(\phi) = \mathcal{L}(\phi) + \lambda \cdot H[q(\beta)] \quad (75)$$

For $q(\beta) = \mathcal{N}(\boldsymbol{\mu}_\beta, \Sigma_\beta)$, the Gaussian entropy is:

$$H[q(\beta)] = \frac{1}{2} \log \det(2\pi e \Sigma_\beta) = \frac{p}{2} (1 + \log 2\pi) + \frac{1}{2} \log \det(\Sigma_\beta) \quad (76)$$

The regularised gradient with respect to Σ_β gains an additional term:

$$\nabla_{\Sigma_\beta} \mathcal{L}_{\text{reg}} = \nabla_{\Sigma_\beta} \mathcal{L} + \frac{\lambda}{2} \Sigma_\beta^{-1} \quad (77)$$

This directly rewards larger Σ_β , counteracting the collapse tendency. The hyperparameter $\lambda > 0$ controls the strength of the correction.

This is the most principled of the three approaches and is directly amenable to the gradient-based optimisation methods (gradient ascent, Newton, BFGS) already implemented for this project.

Selected Approach for This Project

Entropy regularisation (Approach 3) will be implemented and compared against the unregularised ELBO. The regularisation parameter λ will be treated as a hyperparameter and tuned by comparing posterior interval coverage against the MCMC reference. Results will be presented as a secondary comparison alongside the main method comparison.

Summary

Mathematical foundations are complete with all algebraic steps documented. The implementation phase can now proceed systematically: CAVI baseline for validation, then gradient ascent, Newton's method, and BFGS, culminating in empirical comparison to determine which optimisation method performs best for variational inference.

All numerical specifications (Cholesky transforms, line search, regularisation, convergence criteria) are ready for direct translation to code. The final report will reference these detailed derivations without rederivation.